量化交易进阶:第65天 – 实盘交易接入（掘金/聚宽/Tushare Pro 实盘接口演示）（进阶阶段·回测框架篇）

原创 逻辑与流 逻辑与流 2026年2月13日 18:19 北京 听全文

本文系统讲解实盘交易接入的核心架构与主流接口。对比研究系统与实盘系统的本质差异,解析实盘系统七大核心模块,明确 Day62~64 所学属于信号+风控、今日接入订单+成交+账户。详解掘金、聚宽、Tushare Pro 三种接口的定位与适用场景,梳理实盘流程八步核心逻辑,给出平台无关的 LiveTradingEngine 最小框架与掘金/聚宽示例代码。揭示实盘与回测三大差异(延迟、部分成交、账户状态不同步)及幂等机制与状态校验的必要性,指出 research/ 与 live/ 分离的正确姿势及实盘前五条必做测试。

一、实盘系统架构（先讲清楚结构）

你现在的研究系统是：

```
历史数据 → 因子 → 权重 → 回测 → 净值
```

实盘系统是完全不同的：

```
实时数据 → 信号 → 风控 → 下单 → 交易所 → 成交回报 → 账户更新
```

区别在于：

回测	实盘
可以用未来数据（不小心）	绝对不能
成交价是假设	成交价真实
滑点可控	滑点不可控
延迟忽略	延迟致命

二、实盘系统核心模块（你必须理解）

一个标准实盘系统包含：

1. 数据模块
2. 信号模块
3. 风控模块
4. 订单模块
5. 成交回报模块
6. 账户状态模块

7. 日志与监控模块

你在 Day62~64 学的东西，其实都属于：

信号 + 风控

今天我们接入：

订单 + 成交 + 账户

三、三种实盘接口对比

1 掘金（GM / GmTrade）

- 专业量化接口
- 支持实盘
- API 完整
- 延迟低
- 支持多账户

适合：真正要跑实盘的人

2 聚宽（JoinQuant）

- 云端策略
- 支持模拟盘
- 支持实盘（券商对接）



- 易用

适合：快速部署 + 云端运行

3 Tushare Pro

注意：

Tushare 主要是数据接口，不是交易接口。

你不能直接用 Tushare 下单。

但可以：

- 用 Tushare 拉实时行情
- 自己对接券商 API

四、实盘流程核心逻辑

真实交易流程是这样的：

1. 到达调仓时间
2. 获取最新行情
3. 计算目标权重
4. 获取当前持仓
5. 计算净下单
6. 发送订单
7. 等待成交回报
8. 更新账户

五、实盘最小框架示例（通用版）

我们先写一个「平台无关」的框架：

```
class LiveTradingEngine:
    def __init__(self, broker):
        self.broker = broker
        self.positions = {}
        self.cash = 0

    def update_account(self):
        account = self.broker.get_account()
        self.cash = account["cash"]
        self.positions = account["positions"]

    def generate_target_weights(self, market_data):
        # 这里接入你 Day62 多策略系统
        return {"600519.SH": 0.2, "000858.SZ": 0.15}

    def rebalance(self, market_data):
        self.update_account()

        target_w = self.generate_target_weights(market_data)

        total_asset = self.cash + sum(
            pos["market_value"] for pos in self.positions.values()
        )
```

```
        for code, weight in target_w.items():
            target_value = total_asset * weight
            current_value = self.positions.get(code, {}).get("market_value", 0)
            diff = target_value - current_value

            if abs(diff) > 1000:
                price = market_data[code]["price"]
                volume = int(diff / price / 100) * 100
                if volume != 0:
                    self.broker.order(code, volume)
```

六、掘金 API 示例

示例（简化版）：

```
from gm.api import *

def init(context):
    context.symbols = ['SHSE.600519']

def on_bar(context, bars):
    for bar in bars:
        if bar.symbol == 'SHSE.600519':
            order_volume(
                symbol='SHSE.600519',
                volume=100,
                side=OrderSide_Buy,
                order_type=OrderType_Market,
                position_effect=PositionEffect_Open
            )

run(strategy_id='你的ID',
    filename='strategy.py',
    mode=MODE_LIVE,
    token='你的token')
```

七、聚宽示例

```python
def initialize(context):
    set_benchmark('000300.XSHG')
    set_option('use_real_price', True)
    context.stock = '600519.XSHG'

def handle_data(context, data):
    if not context.portfolio.positions:
        order_target_percent(context.stock, 0.1)
```

八、实盘与回测最大差异

1 延迟

回测：今天收盘算 → 明天开盘成交（假设）

实盘：



- 9:30:01 下单
- 9:30:01.300 进入券商
- 9:30:02.100 成交

行情已变。

2 部分成交

回测：假设全部成交

实盘：只成交一半，剩余挂单

3 账户状态不同步

- 成交回报延迟
- API 调用失败
- 网络波动

必须设计：**幂等机制 + 状态校验**

九、实盘接入正确姿势

你现在的系统应分为两部分：

```
research/
    backtest_engine.py
    factor_models.py

live/
    signal_engine.py
    broker_adapter.py
    execution_manager.py
```

十、实盘前的「必做测试」

1. 模拟盘跑至少 1 个月
2. 打印所有订单与成交日志
3. 对比理论持仓 vs 实际持仓
4. 校验滑点模型
5. 统计未成交率

十一、Day65 总结

回测是「数学问题」，实盘是「系统工程问题」。

真正难的不是因子，而是：

- 延迟
- 流动性
- 成交管理
- 账户同步

结论

回测是「数学问题」，实盘是「系统工程问题」。真正难的不是因子，而是：延迟、流动性、成交管理、账户同步。

量化交易入门指南 · 目录 ≡

〈上一篇
量化交易进阶:第64天 – 多标的管理系统（多股票组合 + 多策略组合管理）(进阶阶段·回...

下一篇 〉
量化交易进阶:第66天 – 实盘风控模块（止损止盈、仓位上限、信号延迟）(进阶阶段·回...



开局一人一狗，无限探索资源，守护人类最后文明！

无尽冬日-3亿玩家力荐

进入小游戏